

```
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2
```

```
import faiss
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM
from PyPDF2 import PdfReader
from google.colab import files
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cpu
```

```
enc_name = "sentence-transformers/all-MiniLM-L6-v2"
enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)
enc_model = AutoModel.from_pretrained(enc_name).to(device)
enc_model.eval()
```

```
Loading weights: 100% 103/103 [00:00<00:00, 162.26it/s, Materializing param=pooler.dense.weight]
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
```

```
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |
```

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 384, padding_idx=0)
    (position_embeddings): Embedding(512, 384)
    (token_type_embeddings): Embedding(2, 384)
    (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-5): 6 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=384, out_features=384, bias=True)
            (key): Linear(in_features=384, out_features=384, bias=True)
            (value): Linear(in_features=384, out_features=384, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=384, out_features=384, bias=True)
            (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=384, out_features=1536, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=1536, out_features=384, bias=True)
          (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=384, out_features=384, bias=True)
    (activation): Tanh()
  )
)
```

```
llm_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

llm_tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm_tokenizer.pad_token = llm_tokenizer.eos_token

llm_model = AutoModelForCausalLM.from_pretrained(
  llm_name,
  torch_dtype=torch.float16 if device == "cuda" else torch.float32,
  device_map="auto"
)
```

```
llm_model.eval()
```

Loading weights: 100%

201/201 [00:10<00:00, 13.84it/s, Materializing param=model.norm.weight]

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 2048)
    (layers): ModuleList(
      (0-21): 22 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)
```

```
uploaded = files.upload()
pdf_path = list(uploaded.keys())[0]
```

 1706.03762v7.pdf

1706.03762v7.pdf(application/pdf) - 2215244 bytes, last modified: 25/03/2026 - 100% done
Saving 1706.03762v7.pdf to 1706.03762v7 (1).pdf

```
def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text += page_text + "\n"
    return text
```

```
def chunk_text(text, chunk_size=150, overlap=30):
    words = text.split()
    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap

    return chunks
```

```
def encode_chunks(chunks, batch_size=8):
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(chunks), batch_size):
            batch = chunks[i:i + batch_size]

            tokens = enc_tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = enc_model(**tokens)
            embeddings = outputs.last_hidden_state.mean(dim=1)

            all_embeddings.append(
                embeddings.cpu().numpy().astype("float32")
            )
```

```
return np.vstack(all_embeddings)
```

```
raw_text = extract_text_from_pdf(pdf_path)

doc_chunks = chunk_text(raw_text)
doc_embeddings = encode_chunks(doc_chunks)

faiss.normalize_L2(doc_embeddings)

dim = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dim)
index.add(doc_embeddings)

print("Total chunks indexed:", len(doc_chunks))
```

Total chunks indexed: 50

```
def retrieve_context(query, top_k=3):

    with torch.no_grad():
        tokens = enc_tokenizer(
            [query],
            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to(device)

        outputs = enc_model(**tokens)
        q_emb = outputs.last_hidden_state.mean(dim=1)

        q_emb = q_emb.cpu().numpy().astype("float32")
        faiss.normalize_L2(q_emb)

    scores, indices = index.search(q_emb, top_k)

    return [doc_chunks[i] for i in indices[0]]
```

```
def generate_rag_answer(question, top_k=3, max_new_tokens=200):

    contexts = retrieve_context(question, top_k)
    context_text = "\n\n".join(contexts)

    # IMPORTANT: limit context length to avoid overflow
    context_text = context_text[:1500]

    prompt = f"""
You are an assistant that answers ONLY using the given context.

Context:
{context_text}

Question: {question}
Answer:
"""

    inputs = llm_tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=1024
    ).to(device)

    with torch.no_grad():
        output_ids = llm_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    output_text = llm_tokenizer.decode(
        output_ids[0],
        skip_special_tokens=True
    )

    return output_text.split("Answer: ")[-1].strip()
```

```
question = "What is the main contribution of this paper?"
answer = generate_rag_answer(question)
```

```
print("\nQuestion:", question)
print("\nAnswer:", answer)
```

Question: What is the main contribution of this paper?

Answer: The main contribution of this paper is the development of a new architecture, the Transformer, which is a transforme